Nixar Core Developer Integration Guide

Table of Contents

Introduction

This document provides information to the developers who want to integrate the Nixar api to their development environment using the **C Interface** wrappers or the **Rust Api**. The **C Interface** wrappers are written for **Java**, **Swift**, **Python** and **ReactJS**. Currently **WASM** (for **NodeJS**) are on the TODO list.

Who Should Read This Document

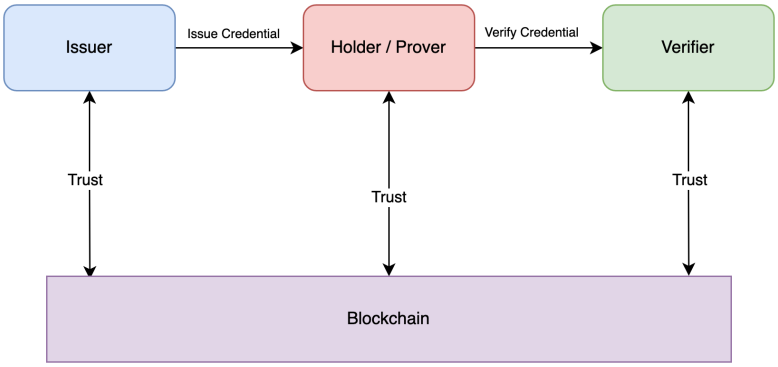
If you study on SSID and Distributed ID blockchains, have interest in creating blockchain agnostic SSID solutions, you already know about SSID and you are interested in developing business applications using the nixar-core library, this document is for you.

Who Should *NOT* Read This Document

Well it wouldn't be so polite to ask people not to read this document; however, if you have one of the following aims: **i) you** seek for theoretical information about the SSID, **ii)** you want to learn more about blockchain, ii) you need to elaborate and research on the business use cases of SSID, then this document is *NOT* for you.

A tiny intro to SSID

The most fundemental use case of the SSID is given below. The main actors of the use case are **Issuer**, * Holder/Prover* and **Verifier**.



In most plain words, an **Issuer** creates *credentials* (i.e. identity information for an entity) for the **Holder** who in return saves these credentials in her *wallet*.

In the next step the **Holder** who **holds** her credentials becomes a , and **proves** (as a **Prover**) these credentials to a **Verifier** (e.g. a *Service Provider*)

While in the above scenarios the roles **Holder** and **Prover** are used interchangeably, in fact an entity in the SSID context might have multiple roles simultaneously (e.g. an issuer might become a prover in another business scenario).

NOTE	We will use the term **Prover** for both Holder and Prover roles in the rest of the document#

An entity in SSID is context is represented with an **Agent**, which is mostly a piece of software that can act on behalf of the entity. *For example, Lucy 's digital agent, on her favorite android device, might create proof from her digital wallet and send it to her schools agent (verifier) in order to submit her homework to the homework submission system.*

Prerequisites

Downloading the binaries

The **nixar-core** library can be built for all popular platforms. The list of the existing precompiled binaries and where to find them is given below

Platform/Resource	Link
Api Docs (Kickstart (this document) and c interfaces)	https://bag.org.tr/proje/nixar_shared/nixar_api_docs
OsX	https://bag.org.tr/proje/nixar_shared/nixar_api_osx
IOS XCFramework	https://bag.org.tr/proje/nixar_shared/cnixarapi.xcframework
IOS Swift Wrapper	https://bag.org.tr/proje/nixar_shared/nixar_api_ios
Linux	https://bag.org.tr/proje/nixar_shared/nixar_api_linux *
Android	https://bag.org.tr/proje/nixar_shared/nixar_api_android
Win64	https://bag.org.tr/proje/nixar_shared/nixar_api_win64 *

The binaries need to be discoverable to your development environment in order for your application to work with the nixar-core library. For example on *OsX* , the binaries **libnixar_core.dylib**, **libcrypto.1.1.dylib**, **libssl.1.1.dylib**, **libzmq.5.dylib**. Please not that you can also use compatible versions of the dependent libraries (i.e. OpenSSL, LibZmq and LibSodium) that preinstalled on your environment. See following sections for more details.

Android: Notes for users

How to include in Android Projects

Please clone or download the java library and install using **maven** and **JDK8**.

```
> mvn -DskipTests=true clean install
```

Then to include the library in your gradle project, please don't forget to add `mavenLocal()` to your `build.gradle` file.

```
allprojects {
  repositories {
    mavenLocal()
    //others...
    google()
    jcenter()
  }
}
```

NOTE	There are two places that `mavenLocal()` can be written, one of them is `buildscript` and the other is `allprojects`. Please ignore the one for `buildscript` and write `mavenLocal` to `allprojects` as shown above.

How To Include Native Libs

In order to include `libnixar_core.so` in your project clone or download this repository. Copy files to your Apk's source path as follows:

```
app/
  build
  build.gradle
  proguard-rules.pro
  src/
    main/
     jniLibs/
      java/
        ...
      armv64-v8a/
        libc++_shared.so
        libjnidispatch.so
        libnixar_core.so
      armeabi-v7a/
        libc++_shared.so
        libjnidispatch.so
        libnixar_core.so
```

Enabling nixar-core logs

By default nixar-core will be able to log `INFO`, `WARN` and `ERROR`. In order to enable `DEBUG` and `VERBOSE` please set this property via `adb` as below.

```
  #For verbose logging
  adb shell setprop log.tag.libnixar-core VERBOSE
  #For verbose logging
  adb shell setprop log.tag.libnixar-core DEBUG
```

Choosing Nixar-Home Directory

In the most recent android versions, writing to certain locations has been highly restricted. Therefore, developers might get file permission errors.

By default, *Nixar* will try to write its content to `EXTERNAL_STORAGE`. But it is a bit confusing and complicated to allow the same access on all devices.

Due to this fact, the most recent version of *Nixar* includes a new api that allows the end users to set the *Nixar* home directory.

The sample code snippet below shows how to use that:

```
NixarAgentFactory.initLogging(Level.TRACE);
//set nixar home path manually to data dir.
NixarAgentFactory.setNixarHomePath(getApplicationContext().getDataDir().getAbsolutePat
h());
NixarAgent nixarAgent = NixarAgent.createAgent(...
```

Please note that this call must take place before any agent manipulation.

Android Kickstart

An android kickstart project has been created here:

https://bag.org.tr/proje/nixar_shared/android-kickstart

Enable Cleartext traffix to `http` networks

1. In order to connect to bcovrin network and download the genesis file automatically from http://test.bcovrin.vonx.io/genesis, you need to enable cleartext traffic:

   AndroidManifest.xml

   ```
   <application
          ...
          android:usesCleartextTraffic="true"
          ...>
   ```

Otherwise you can manually download the genesis file and feed to the app.

Building from source (Optional)

The nixar-core library needs **Rust**, **libzmq**, **OpenSSL** and **LibSodium** if you want to build from source.

Rust and Cargo

Please install Rust [Rust](https://www.rust-lang.org/tools/install) and [Cargo](https://doc.rust-lang.org/cargo/getting-started/installation.html) in order to be able to build the codes from the source. You need to access the repository addresses of the nixar-core source code as well as its dependent

OpenSSL

The library needs [OpenSSL](https://www.openssl.org/) V1.1 which is already included in the repository (libssl.1.1.dylib and libcrypto.1.1.dylib). Feel free to use it or install it your system via `brew`

ZeroMQ

[ZeroMQ](https://zeromq.org/) is used for communicating with the Hyperledger Indy networks. We have included a precompiled dylib (libzmq.5.dylib) but you can also obtain it from outside

Libsodium

Please follow Libsodium installation instructions are provided [here](https://doc.libsodium.org/installation). Nixar-core is compatible with [libsodium 1.0.18 -RELEASE](https://github.com/jedisct1/libsodium/tree/1.0.18-RELEASE) or newer.

Library Samples

In this chapter we provide samples for the agent creation, connection manipulation and generic anoncreds use cases (e.g `create agent`, `issue`, `store` and `verify`)

NOTE	You can see the full source codes for the wrappers and example codes from the below links:

Language	Address
Rust	*TBD*
Java	https://bag.org.tr/proje/nixar_shared/nixar_api_java
Python	https://bag.org.tr/proje/nixar_shared/nixar_api_python
Swift	https://bag.org.tr/proje/nixar_shared/nixar_api_ios

Create Agent

The main actor of the nixar-core library is an *agent* object. In order to start using the library you need to create an agent object.

Java

```
AgentInitParams agentInitParams=new
AgentInitParams("my_issuer","ENDORSER","000000000000000000000TUBISSUER1","./genesis-
bcovrin.txn");
    NixarAgent issuer=NixarAgent.createAgent(agentInitParams,()->{
    System.out.println("Callllllll");
    return"123456".toCharArray();
    });
```

Rust

```
fn main() {
  let agent_init_params = AgentInitParams {
    alias: agent_name.clone(),
    role: Some("ENDORSER".to_string()),
    seed: Some("000000000000000000000TUBISSUER1".to_string()),
    genesis_path: genesis.clone(),
  };
  let issuer_agent = NixarAgent::create(agent_init_params, Box::new(||
"123456".to_string()))?;
}
```

Python

```
issuer = NixarApi('genesis.txn')
issuer.create_agent('python_issuer', 'ENDORSER', '12345',
                '000000000000000000000000TUBISSUER1')
```

Swift

```
let passworCallback = { () in
    return "12345"
}

let agentInitParams = AgentInitParams(alias: "tub_ios_issuer",
                                      role: "ENDORSER",
                                      seed: "000000000000000000000000TUBIOSUER1",
                                      genesisPath: genesisPath);

var issuer : NixarAgent = try
NixarAgentFactory.createAgent(agentInitParams:agentInitParams,
passwordCallback:passworCallback)
```

In the above code, we first create and `AgentInitParams` object/json string. We have to provide an `agent_name` that will be the *name of our agent*. We shall then refer to our agent (e.g. open it) with the same name. A `role` is optional for an agent. Mostly mobile device users (i.e. Prover) will not need a role; because one wouldn't need a `role` in order to perform `READ` operations on the ledger.

However, if you are an issuer your role has to at least be an `ENDORSER` in order to add schemas and credential definitions to the *ledger* as well as revoking credentials and so on. In other words, to perform `WRITE` operations on the ledger you need to have your `NYM` registered to the ledger and you `need` a role. In that case another agent with e `TRUSTEE` or `ENDORSER` role must have added your agent to the ledger.

Otherwise you can skip the role and set it to `None/null`.

Manipulating Connections Between Agents

In order for agents to communicate via a did channel, their connection information must previously have been established. This scenario is called creating a connection between agents. The steps are given below.

In below samples we assume that we want to connect two agents, namely *Agent1* and *Agent2*

Agent 2 Create Connection Invitation

Assuming that your agent is `agent1` and you want to connecto to `agent2`, you need an invitation that has been created by `agent2`. For that `agent2` should create an invitation.

There are two types of invitations, i) *public invitation*, ii) *private invitation*. In the first case, the invitation contains the information about the **did** of the agent. In the second case, the invitation contains additinally the endpoint (mostly HTTP) of the agent so that the other agent can discover its location and connects to it. In the case of a *public invitation*, the connecting agent needs to resolve the endpoint and connection keys from the *ledger*.

Below is the sample code for creating a connection invitation or getting the public invitation. The owner of the agent is supposed to share this information with the clients, via means like publishing on the web, QR codes and so on.

Java

```
// Agent 2 get public invitation
invitation = agent2.getPublicInvitation();
```

Rust

```
// for a local invitation
let invitation = agent2.connection_manager.create_invitation("my local invitation",
Some("http://myserver/agent".to_string()), false).unwrap()
//for the public invitation
let invitation = agent2.connection_manager.get_public_invitation().unwrap()
```

Python

```
# Agent A get public invitation
invitation = agent2.connection_get_public_invitation()
```

Swift

```
// Swift
```

Agent-1 Create Connection Request

Having received the invitation from *agen2*, our agent (*agent1*) needs to process this invitation and create a connection request from it. As stated previously, if the invitation is public, the endpoint and recipient keys of the *agent2* are resolved from the ledger by *nixar-core* automatically.

Java

```
// Agent 1 receive invitation and create connection request
requestMessage = prover.createConnectionRequest(invitation);
```

Rust

```
let request_message = prover.connection_manager
   .create_connection_request(&invitation).unwrap()
```

Python

```
# Agent 1 receive invitation and create connection request
request_message = prover.connection_create_request(invitation)
```

Swift

```
//TODO
```

The resulting `request_message` is the *encrypted* connection request message created by *agent1*. The encryption is performed using *did-comm* and only *agent2* can open the message.

Agent-2 Accept Connection Request

Agent 2 receives the *encrypted connection request* via means decided by the business layer (e.g. HTTP, e-mail, AS4 ...). Here we assume that *agent2* has received the message; subsequently processes it, accepts the connection request and creates an *encrypted connection response*.

Java

```
//Java
// Agent 2 receives connection request and create connection response
responseMessage = issuer.acceptConnectionRequest(requestMessage, null);
```

Rust

```
let response_message = issuer_agent.connection_manager
    .accept_connection_request(message, Some("connection with prover")).unwrap();
```

Python

```
# Python
# Agent 2 receives connection request and create connection response
response_message = issuer.connection_accept_request(conn_req, None)
```

Swift

```
//TODO
```

The second parameter is an alias that *agent2* can optionally set to the underlying connection object.

The *response_message* created by *agent2* is also encrypted and can only be opened by *agent1*.

Agent-1 Accept Connection Response

As the last step of connection creation, *agent1* needs to process the *response_message* created by *agent2* in the previous step. Below is the code.

Java

```
//Agent 1 receives connection response and finalize connection creation
agent1.connectionAcceptResponse(responseMessage);
```

Rust

```
agent1.connection_manager.accept_connection_response(response_message)
```

Python

```
# Agent 1 receives connection response and finalize connection creation
prover.connection_accept_response(response_message)
```

Swift

```
//TODO
```

At the end of this sequence of messages `agent1` and `agent2` have a common pairwise did and are ready to communicate further via this connection using *DidComm* protocol

Encrypting - Decrypting Messages Between Agents

After the creation of connection between two agents, the rest of the communication for sensitive messages needs to be performed via a secure channel based on *DidCOMM*. A sample code for encrypting and decrypting a credential is given below:

Java

```
//TODO
```

Rust

```
//TODO
```

Python

```
# Agent A Create Simpele Message
from_did = agent_a_conns[0]["my_did"]
their_did = agent_a_conns[0]["their_did"]
message = "hello agent A"
encrypted_message = issuer.connection_encrypt(
from_did, their_did, NixarMessageType.MESSAGE_TYPE_SIMPLE, message)
logging.info("Agent A create message, encrypted_message:"
+json.dumps(encrypted_message))

# Agent B receive message and decrypt
decrypted_message = prover.connection_decrypt(encrypted_message)
logging.info("Agent B decrypt message, decrypted_message:" +
json.dumps(decrypted_message))
```

Swift

```
//TODO
```

Issue-Store-Verify-Revoke Credentials

After communication creation between different agents, the famous trinity of issue-prove-verify can be achieved. This is the basic and most generic scenario of SSID.

Please refer to [Create Agent](#Create%20Agent) section for agent creation and [Agent Connection](#connect-to-another-agent-agent1-agent2) for connection between agents.

Please also note that the message flow between the `issuer`, `prover` and `verifier` needs to be encrypted using *DidComm*. Although the examples below do not contain an encryption example, we will remind about this frequently and give an ecryption-decryption example in the end of the document.

Issuer Creates Schema and Credential Definition

A prerequisite of creating credentials for another entity is to create a *schema* and a *credential definition* for the target credential.

A *schema* defines which attributes will exist in a credential. Its basically a list of attributes with additional schema name and ID.

A *credential definition* cryptologically defines *how an issuer will issue a credential based on the schema*.

The issuer creates the schema and the credential definition and writes them to the ledger.

Java

```
String schemaId = issuer.createSchema("java_schema21", new
HashSet<>(Arrays.asList("name", "surname", "age", "gender")), "2.0");
String credDefId = issuer.createCredentialDefinition(schemaId, true,
NixarNativeInterface.CredDefIssuanceType.IssuanceByDemand);
```

Rust

```
//Create schema
let schema_name = "myschema".to_string();
let schema_version = "2.0".to_string();

let attribute_set = {
  let mut set = HashSet::new();
  set.insert("name".to_string());
  set.insert("surname".to_string());
  set.insert("age".to_string());
  set.insert("gender".to_string());
  set
};
let res = issuer_agent.issuer.create_schema(schema_name, schema_version,
attribute_set.into());
let api_result = ApiResult(res);
println!("Schema created, schema_id: {:}", jsn!(&api_result));
let schema_id = api_result.0.unwrap();
//Create credential definition
let cred_def_tag = "bilgem".to_string();
let signature_type = String::from("CL");
let res = issuer_agent.issuer.create_credential_definition(schema_id, cred_def_tag,
signature_type, true, Some(ISSUANCE_ON_DEMAND));
let api_result = ApiResult(res);
println!("Cred Def created, cred_def_id: {:}", jsn!(&api_result));
let cred_def_id = api_result.0.unwrap();
```

Python

```
schema_name = 'python_schema'
schema_version = '2.0'
attribute_set = '["name","surname","age","gender"]'

schema_id = issuer.issuer_create_schema(
    schema_name, attribute_set, schema_version)

logging.info('Schame created, schema_id: ' + schema_id)

cred_def_id = issuer.issuer_create_credential_definition(schema_id, True,
CredDefIssuanceType.ISSUANCE_ON_DEMAND)

logging.info('Credential definition created, cred_def_id: ' + cred_def_id)
```

Swift

```
let schemaName = "swift_schema"
let attributeSet = ["name", "surname", "age", "gender"]
let schemaVersion = "2.0"

let schemaId = try issuer.createSchema(schemaName: schemaName, attributeSet:
attributeSet, schemaVersion: schemaVersion)

 let credDefId = try issuer.createCredentialDefinition(schemaId: schemaId,
isRevocable: true, issuanceType: .ISSUANCE_ON_DEMAND, maxCredNum: 20)
```

Issuer Creates Credential Offer

The second step of credential issuance is to create an offer for a *prover*. This use case can be triggered by a client (i.e. a prover requests a credential from an issuer) and issuer creates an offer as below.

Java

```
Random random = new Random();
byte bytes[] = new byte[10];
random.nextBytes(bytes);

BigInteger bIssuerNonce = new BigInteger(1, bytes);
String issuerNonce = bIssuerNonce.toString(10);

CredentialTypes.CredentialOffer credOffer = issuer.createCredentialOffer(credDefId,
issuerNonce);
```

Rust

```
let issuer_nonce = Nonce::new()?;

let res = issuer.issuer.create_credential_offer(cred_def_id, issuer_nonce);
let api_result = ApiResult(res);
println!("Cred Offer created, cred_offer: {:}", jsn!(&api_result));
let cred_offer = api_result.0.unwrap();
```

Python

```
issuer_nonce = str(random.getrandbits(80))

cred_offer = issuer.issuer_create_credential_offer(cred_def_id, issuer_nonce)
```

Swift

```
let issuerNonce = BigUInt.randomInteger(withExactWidth: 80)

let offer = try issuer.createCredentialOffer(credDefId: credDefId,
issuerNonce:issuerNonce);
```

Plase don't forget to ensure message security as described in [Encrypting - Decrypting Messages Between Agents](#encrypting---decrypting-messages-between-agents)

Prover Creates Credential Request

Having received an *offer* from the issuer, the `prover` needs to create a `credential_request` from the offer. The sample code is below.

Java

```
CredentialTypes.CredentialRequest credReq = prover.createCredentialRequest(credOffer);
```

Rust

```
let res = prover.prover.create_credential_request(&cred_offer);
let api_result = ApiResult(res);
println!("Cred Request created, cred_req: {:}", jsn!(&api_result));
let cred_req = api_result.0.unwrap();
```

Python

```
cred_req = prover.prover_create_credential_request(cred_offer)
```

Swift

```
let request = try prover.createCredentialRequest(credOffer: offer)
```

Prover send credential the credential request and the issuer nonce which is used while creating credential offer to issuer to create credential.

Plase don't forget to ensure message security as described in [Encrypting - Decrypting Messages Between Agents](#encrypting---decrypting-messages-between-agents)

Issuer Creates Credential

The `prover` creates and sends a credential request to the issuer in the previous step. In this step, the `issuer` will process the credential request and create a credential for `prover`

Java

```
 Map<String, CredentialTypes.AttributeValues> credValues = new HashMap<>();
credValues.put("name", new CredentialTypes.AttributeValues("Mehmet"));
credValues.put("surname", new CredentialTypes.AttributeValues("Öztürk"));
credValues.put("age", new CredentialTypes.AttributeValues("30"));
credValues.put("gender", new CredentialTypes.AttributeValues("male"));

CredentialTypes.CredentialInfo issuedCredInfo = issuer.createCredential(credReq,
JsonHelper.toJson(credValues), issuerNonce);
```

Rust

```
let mut values: HashMap<String, AttributeValues> = HashMap::new();
values.insert(String::from("name"), AttributeValues { raw: String::from("Mehmet"),
encoded: String::from("1701733747") });
values.insert(String::from("surname"), AttributeValues { raw: String::from("Öztürk"),
encoded: String::from("521660491122") });
values.insert(String::from("age"), AttributeValues { raw: String::from("30"), encoded:
String::from("30") });
values.insert(String::from("gender"), AttributeValues { raw: String::from("male"),
encoded: String::from("1835101285") });

let cred_values = CredentialValues(values);
println!("CredentialValues {}", jsn_pretty!(&cred_values));

let res = issuer.issuer.create_credential(cred_req, cred_values, issuer_nonce);
let api_result = ApiResult(res);
println!("Credential created, cred: { }", jsn!(&api_result));

let (cred, cred_rev_id) = api_result.0.unwrap();
```

Python

```
cred_values =
'{"name":{"raw":"Mehmet","encoded":"1701733747"},"surname":{"raw":"Öztürk","encoded":"
521660491122"},"age":{"raw":"30","encoded":"30"},"gender":{"raw":"male","encoded":"183
5101285"}}'
issued_cred_info = issuer.issuer_create_credential(
    cred_req, cred_values, issuer_nonce)
```

Swift

```
var credValues = [String: AttributeValues]()
credValues["name"] = AttributeValues(raw: "Mehmet");
credValues["surname"] = AttributeValues(raw: "Öztürk");
credValues["age"] = AttributeValues(raw: "30");
credValues["gender"] = AttributeValues(raw: "male");

let credential = try issuer.createCredential(credRequest: request, credValues:
JSONHelper.encode(credValues), issuerNonce:issuerNonce)
```

Plase don't forget to ensure message security as described in [Encrypting - Decrypting Messages Between Agents](#encrypting---decrypting-messages-between-agents)

Prover Stores Credential

Having received the new credentials from the `issuer`, the `prover` agent needs to verify and save those credentials to its wallet. This is achieved as below:

Java

```
boolean storeCred = prover.storeCredential(null, issuedCredInfo.getCredential());
```

Rust

```
let res = prover.prover.store_credential(None, &cred);
let api_result = ApiResult(res);
println!("Credential stored, is_cred_stored: { }", jsn!(&api_result));
```

Python

```
store_cred = prover.prover_store_credential(None, cred)
```

Swift

```
let stored = try prover.storeCredential(credId: "myCredential", credential:
credential.credential)
```

Plase don't forget to ensure message security as described in [Encrypting - Decrypting Messages Between Agents](#encrypting---decrypting-messages-between-agents)

Verifier Creates Presentation Request (Proof Request)

Verifier is supposed to create a presentation request (i.e. e proof request) in order to provide service to an entity (i.e. a prover). A presentation request is a json object that contains information and metadata about what data the verifier needs for this specific verification scenario.

Java

```
pres_req = '{"name": "IdentityPR", "version": "2.0", "nonce": "59240876",
"requestedAttributes": {"attribute1_referent": {"name": "name"},
"attribute2_referent": {"name": "surname"}, "attribute3_referent": {"name": "gender"},
"attribute4_referent": {"name": "hobies"}}, "requestedPredicates":
{"predicate1_referent": {"name": "age", "p_type": ">", "p_value": 10}}}'
```

Rust

```
pres_req = '{"name": "IdentityPR", "version": "2.0", "nonce": "59240876",
"requestedAttributes": {"attribute1_referent": {"name": "name"},
"attribute2_referent": {"name": "surname"}, "attribute3_referent": {"name": "gender"},
"attribute4_referent": {"name": "hobies"}}, "requestedPredicates":
{"predicate1_referent": {"name": "age", "p_type": ">", "p_value": 10}}}'
```

Python

```
pres_req = '{"name": "IdentityPR", "version": "2.0", "nonce": "59240876",
"requestedAttributes": {"attribute1_referent": {"name": "name"},
"attribute2_referent": {"name": "surname"}, "attribute3_referent": {"name": "gender"},
"attribute4_referent": {"name": "hobies"}}, "requestedPredicates":
{"predicate1_referent": {"name": "age", "p_type": ">", "p_value": 10}}}'
```

Swift

```
pres_req = '{"name": "IdentityPR", "version": "2.0", "nonce": "59240876",
"requestedAttributes": {"attribute1_referent": {"name": "name"},
"attribute2_referent": {"name": "surname"}, "attribute3_referent": {"name": "gender"},
"attribute4_referent": {"name": "hobies"}}, "requestedPredicates":
{"predicate1_referent": {"name": "age", "p_type": ">", "p_value": 10}}}'
```

The json contains what data the prover needs to prove: (name, surname, gender), optional/additional data (that is not issued but provided by the prover) (hobbies), and predicates (age > 10). The `verifier` will send this request to the `prover`.

Plase don't forget to ensure message security as described in [Encrypting - Decrypting Messages Between Agents](#encrypting---decrypting-messages-between-agents)

Prover Creates Presentation (Proof)

The `prover` receives the `presentation_request` from the `verifier` and creates a presentation (a proof) from it using its existing credentials.

Java

```
Map<String, PresentationTypes.PredicateInfo> reqPreds = new HashMap<>();

PresentationTypes.PredicateInfo predicateInfo = new PresentationTypes.PredicateInfo();
predicateInfo.setName("age");
predicateInfo.setP_type(PresentationTypes.PredicateTypes.GT);
predicateInfo.setP_value(10);

reqPreds.put("predicate1_referent", predicateInfo);

presReq.setRequestedPredicates(reqPreds);

// Prover create presentation
String selfAttestedValues = "{\"attribute4_referent\": \"swim\"}";

PresentationTypes.Presentation pres = prover.createPresentation(presReq,
selfAttestedValues);
```

Rust

```
let mut self_attested_values: HashMap<String, String> = HashMap::new();
self_attested_values.insert(String::from("attribute4_referent"),
String::from("Chess"));

let self_values = SelfAttestedValues(self_attested_values);

let res = prover.prover.create_presentation(&presentation_request,
&Some(self_values));
let api_result = ApiResult(res);
println!("Presentation created, presentation: { }", jsn!(&api_result));
let presentation = api_result.0.unwrap();
```

Python

```
# if self attested value exist
self_attested_values = {}
self_attested_values["attribute4_referent"] = "chess"

pres = prover.prover_create_presentation(pres_req, self_attested_values)
```

Swift

```
//if self attested value exist
let selfAttestedValues = {}
selfAttestedValues["attribute4_referent"] = "chess"

var presentation = try prover.createPresentation(presentationRequest: presReq,
selfAttestedValues: selfAttestedValues)
```

Verifier Verifies Presentation (Proof)

Having received the `presentation` from the `prover`, the `verifier` verifies the `presentation` and then decides to give the service.

Java

```
boolean vResult = verifier.verifyPresentation(presReq, pres);
LOGGER.info("verfier verify presentation, result: " + vResult);
```

Rust

```
let res = verifier_agent.verifier.verify_presentation(&presentation, &pres_req);
let api_result = ApiResult(res);
println!("Presentation verified: { }", jsn!(&api_result));

let pr_result = api_result.0.unwrap();
assert_eq!(&pr_result, &true);
```

Python

```
v_result = verifier.verifier_verify_presentation(pres_req, pres)
```

Swift

```
//verifier verifies the presentation
var result = try verifier.verifyPresentation(presentationRequest: presReq,
presentation: presentation)
print("Verified", result)
XCTAssert(result == true)
```

Plase don't forget to ensure message security as described in [Encrypting - Decrypting Messages Between Agents](#encrypting---decrypting-messages-between-agents)

Issuer Revoke Credential

The `issuer` can revoke a credential that it has issued to a `prover` before.

Java

```
issuer.revokeCredential("" + issuedCredInfo.getCredRevIndex(), credDefId);
```

Rust

```
let cred_rev_id = &cred_rev_id.unwrap();
let cred_def_id = cred.cred_def_id.as_ref();

println!("revoke_credential: {}, {}", cred_rev_id.as_str(), cred_def_id);
let res = issuer.issuer.revoke_credential(cred_rev_id, cred_def_id);

let api_result = ApiResult(res);
println!("Revoke Succeeded {:}", jsn!(&api_result));
```

Python

```
r_result = issuer.issuer_revoke_credential(cred_rev_id, cred_def_id)
```

Swift

```
try issuer.revokeCredential(credRevIndex: credential.credRevId!, credDefId: credDefId)
```

Tail Sharing

If the issuer has created a revokable credential definition, the tail information has to be published so that the provers can access it. The issuer can share the tail information whatever way it wants, that is not in our scope.

Here we provide samples on how to get the tails info from the issuer and how to store this information on the prover.

Issuer Read Tail

Java

```
// obtain revocationRegsitryId  from credential definition
String revocationRegistryId =
"TbAqayWyFFK6VN7eebBTf1:4:TbAqayWyFFK6VN7eebBTf1:3:CL:302423:bilgem:CL_ACCUM:bilgem";
final String tail = issuer.getTail(revocationRegistryId);
```

Rust

```
let tail =
issuer.issuer.get_tail("DgFLKNAB7JKupVCUCdX6V5_4_DgFLKNAB7JKupVCUCdX6V5_3_CL_325_bilge
m_CL_ACCUM_bilgem")?;
```

Python

```
revocation_registry_id =
'7bubM6JWgzLey7dcc9dY2V:4:7bubM6JWgzLey7dcc9dY2V:3:CL:304820:bilgem:CL_ACCUM:bilgem'
tail = issuer.issuer_get_tail(revocation_registry_id)
```

Prover Store Tail

Java

```
// obtain revocationRegsitryId  from credential definition
String revocationRegistryId =
"TbAqayWyFFK6VN7eebBTf1:4:TbAqayWyFFK6VN7eebBTf1:3:CL:302423:bilgem:CL_ACCUM:bilgem";
prover.storeTail(revocationRegistryId, tail);
```

Rust

```
prover.prover.store_tail("RbAqayWyFFK6VN7eebBTf1:4:RbAqayWyFFK6VN7eebBTf1:3:CL:302423:
bilgem:CL_ACCUM:bilgem", &tail)?;
```

Python

```
revocation_registry_id =
'7bubM6JWgzLey7dcc9dY2V:4:7bubM6JWgzLey7dcc9dY2V:3:CL:304820:bilgem:CL_ACCUM:bilgem'
prover.prover_store_tail(revocation_registry_id, tail)
```

Swift

```
prover.storeTail(revRegId:
"TbAqayWyFFK6VN7eebBTf1:4:TbAqayWyFFK6VN7eebBTf1:3:CL:302423:bilgem:CL_ACCUM:bilgem",
tail: tail)
```